Tuning the subtree slide

Why it gets stuck, how often it matters, and a fix (beast3#218)

Alexei Drummond

Monday 9 November 2026

Research group meeting

Where this came from

- BEAST 3 release testing (`beast3-test`): 30 analyses built by hand in BEAUti 2.7.9 and BEAUti 3, run in both versions and compared.
- Both BEAUtis start `BactrianSubtreeSlide` at `size="1.0"`. On substitution-scale trees that is about 100× too large (beast3#200).
- Proposed fixes on the table:
  - #201: start the slide at 0.01 instead;
  - #214: `BactrianSubtreeScale`, a scale-invariant variant of the slide.
- Question for today: which start size, or which operator, should BEAUti write?

The slide's size is in the tree's own units

- The slide moves a node up or down by a draw of scale **`size`**, in units of branch length.
- When it tunes, it settles at about the same *relative* size every time:

$$\text{size} \;\approx\; 0.25\text{–}0.45 \times \frac{H}{\sqrt{2n-1}} \qquad (H = \text{root height},\ n = \text{taxa})$$

- Our analyses span 7 orders of magnitude in H, so the right size does too:

Analysis	n	H	tuned size
C18 WAG protein	10	0.25	0.025
C01 JC69 anolis	29	0.19	0.0089
C10 RSV2, tip dates (years)	129	56	0.95
C01, clock fixed at 1×10^{-4}	29	1,926	88
C01, clock fixed at 1×10^{-7}	29	1,926,000	88,000

Why a bad start never recovers

Auto-tuning is a Robbins–Monro update on log(size) after each proposal:

$$\log s_{k+1} \;=\; \log s_k \;+\; \frac{\alpha_k - \alpha^*}{k} \qquad (\alpha^* = 0.3 \text{ for the slide})$$

- The gain $1/k$ shrinks fast: the total distance $\log s$ can travel in N proposals is at most about

  $$\text{up: } 0.7 \ln N \qquad \text{down: } 0.3 \ln N$$

- 3M steps $\approx$ 590,000 slide proposals $\Rightarrow$ at most ≈ 9 in $\log s$, i.e. about 8,000-fold up, 50-fold down.
- Observed: from 1.0 and from 0.01 alike, the size grew at most 8,000-fold. Longer runs barely help.
- The asymmetry comes from the target 0.3: a size that is too small is corrected faster than one that is too large.

Are tall trees real? PhyloData says yes

- 110 published BEAST experiments with trees on `phylodata.com`.
- Mean root age from the site's own metadata:
  - 29 have $H \geq 100$
  - 3 have $H \geq 1000$
- Tall trees come from dating in years with a fixed or calibrated clock.
- Our tallest test analysis: $H \approx 105$ (testEBSP).

Experiment	n	H
linck-2020 (kingfishers, ND2)	38	2,331,785
auderset-2023 (Mixtecan)	110	2,041
auderset-2023	110	1,747
douglas-2025 (cephalopods)	80	499
v-2016 (corals)	579	472
hara-2021	79	270

linck-2020: Yule, clock fixed at 2.9×10^{-8} per year. Its BEAST 2.5 XML doesn't load in BEAST 3, so we reproduce the scale with C01.

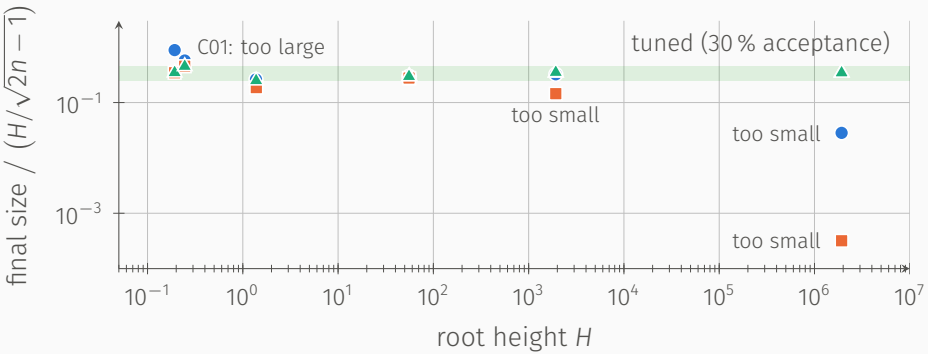
Experiment: the same analysis at three tree scales

C01 (anolis, JC69, Yule, strict clock), unchanged except the fixed clock rate. 3M steps, seeds 1 and 2; both replicates agree in every cell.

Clock rate (H)	Slide from 1.0	Slide from 0.01	Scale operator (#214)
1 (0.19)	stuck large: acc. 1–3 %	tuned: 30 %	tuned: 30 %
1×10^{-4} (1,900)	tuned: 34 %	stuck small: 73 %	tuned: 30 %
1×10^{-7} (1.9×10^{6})	stuck small: 95 %	stuck small: 99.9 %	tuned: 30 %

- #201 (start at 0.01) moves the failure from short trees to tall ones.
- No single start size works for every analysis.

Where the slide ends up



● gain $1/k$, start 1.0 (current BEAUti) ■ gain $1/k$, start 0.01 (#201) ▲ gain $1/k^{0.6}$, start 1.0 or 0.01 (#218)

One point per analysis and set-up (both replicates agree). C26 samples from the prior, so its tree height varies too widely for this scaling; it is left out.

What a stuck slide costs

- The posterior is never wrong: every set-up matches (tall trees are exactly 10^4 and 10^7 times C01's).
- Only mixing suffers, mostly tree height. ESS per million steps (2 replicates summed, trace logged every 100 steps):

Tree	Set-up	likelihood	tree height
$H \approx 0.19$	slide from 1.0, stuck	2753	4211
	slide from 0.01, tuned	3445	6307
$H \approx 1.9 \times 10^6$	slide from 1.0, stuck	3091	3983
	slide from 0.01, stuck	2505	3638
	`BactrianSubtreeScale`	3121	5681

A stuck slide loses about 20–35 % of the ESS. Other tree operators (up-down, uniform, exchange, Wilson–Balding) carry the rest.

Options we considered

Option	Outcome
Start at 0.01 (#201)	Fixes short trees, breaks trees $\gtrsim$ 1000 tall.
`BactrianSubtreeScale` (#214)	Scale-invariant: tunes to 0.19 on every tree. 5–15 % lower ESS than a well-tuned slide.
Start size from the starting tree	Fails exactly where needed: the random starting tree was a few units tall, the posterior tree 1.9 million.
`transform="sqrt"` on the schedule	Gain $1/\sqrt{k}$: the slide tunes from anywhere. But it changes every operator, and $1/\sqrt{k}$ is the boundary case for convergence.
Slide alone in a sub-schedule with `sqrt`	Works today in XML (tuned from both starts, best ESS), but awkward for BEAUti to write.

The fix: a gain exponent (beast3#218)

Gain $1/k^e$ with $0 < e \leq 1$; $e \in (0.5, 1]$ keeps the usual convergence conditions (Andrieu & Thoms 2008). With $e = 0.6$, $\log s$ can travel as $N^{0.4}$ instead of $\ln N$: about 360 instead of 9 in our runs.

- `OperatorSchedule gainExponent`: for every operator in the schedule (1 = today, 0.5 = `sqrt`).
- `Operator gainExponent` input, plus `getDefaultGainExponent()` on the pattern of `targetAcceptanceProbability`. Default: the schedule decides.
- `BactrianSubtreeSlide` defaults to 0.6. No XML or BEAUti change; `gainExponent="1"` restores the old behaviour.

```
<operator spec="kernel.BactrianSubtreeSlide" .../>          now 1/k^0.6
<operator spec="ScaleOperator" ... gainExponent="0.6"/>     one operator
<operatorschedule ... gainExponent="0.6"/>                  all operators
```

Results with #218: the stuck cases

Unchanged XML, #218 build, seeds 1 and 2. All tune to 30 % and match the reference posterior.

Case (stuck with $1/k$)	final size	acc.	likelihood ESS/M	tree height ESS/M
$H \approx 0.19$, from 1.0	0.0089	30 %	3592 (2753)	6482 (4211)
$H \approx 1900$, from 0.01	88	30 %	4672 (3309)	6263 (5639)
$H \approx 1.9 \times 10^6$, from 1.0	88,900	30 %	4189 (3121)	5990 (5681)
$H \approx 1.9 \times 10^6$, from 0.01	88,000	30 %	3489 (3121)	5557 (5681)

In brackets: the $1/k$ slide on the short tree, `BactrianSubtreeScale` on the tall ones. From 1.0 and from 0.01 the slide ends at the same size: the start no longer matters.

Results with #218: analyses that already tuned

Where the $1/k$ slide already tuned from 1.0: does $1/k^{0.6}$ cost anything? Unchanged XML, #218 build, seeds 1 and 2.

Analysis	H	size, $1/k \to 1/k^{0.6}$	acc. $1/k \to 1/k^{0.6}$	ESS ratio, median (range)	posterior
C06 partitions	1.4	0.075 → 0.072	28 → 30 %	0.99 (0.71–1.37)	matched
C10 tip dates	56	1.03 → 1.05	30 → 30 %	0.98 (0.90–1.21)	matched
C13 EBSP*	56	1.03 → 1.02	30 → 30 %	0.98 (0.50–2.43)	matched
C18 protein	0.25	0.032 → 0.025	19 → 30 %	0.99 (0.88–1.10)	matched
C01, $H \approx 1900$	1,926	83 → 89	34 → 30 %	0.97 (0.83–1.00)	matched

- Same tuned size, and no systematic change in mixing: ESS ratio ($1/k^{0.6}$ over $1/k$, every logged column) has median 0.97–0.99. Single columns vary by ±10–30 % either way, which is run-to-run noise with two replicates.

*C13 is compared with the `BactrianSubtreeScale` run, whose build also has the `SampleOffValues` fix; its 272 columns include 129 binary indicators, hence the wide range.

Found along the way

- **EBSP doesn't sample the posterior in BEAST 2.7.9 or BEAST 3 rc2.** `SampleOffValues` had the wrong Hastings ratio: it was accepted 2 % of the time instead of 100 %, and the switched-off population sizes averaged 21 instead of 42. Fixed in beast3#203 (after rc2); beast2#1223 is still open. C13's BEAST 2 vs BEAST 3 "match" compared two runs with the same bug.
- **A constant 0.1 in the prior** (C18): BEAUti 3 writes the gamma-shape prior as a truncated Exponential(1) above 0.1, normalised by $e^{0.1}$; BEAUti 2.7.9 only bounds the parameter. The posterior is the same.
- $1/k$ **can stick on other operators too.** The schedule-wide `gainExponent` lets an analysis try 0.6 for every operator.

Proposal and open questions

Proposal for 2.8.0

- Merge #218: `BactrianSubtreeSlide` tunes with $1/k^{0.6}$ by default.
- Keep #201's start of 0.01: with target 0.3 the size moves up $0.7/k$ but down only $0.3/k$, so it is better to start too small than too large.
- Merge #214 as an extra operator; the two don't conflict.

Open questions

- Should the schedule default become 0.6 for every operator? Needs a broader test, including the relaxed and random local clock analyses (C14–C17).
- Is 0.6 the right exponent? 0.5–0.7 all travel far enough; the trade-off is late-run jitter in the size.
- Five replicates per set-up to pin down the 5–15 % ESS differences.

Where everything is

- Runs, comparisons and write-up: `beast3-test` repo, `reports/beauti-manual.md`, `reports/summary/`, `beast3/log/`. Tree files: Dropbox `beast3-test`.
- Scripts: `scripts/compare.py`, `scripts/slide_variants.py`.
- Issues and PRs: beast3#200 (problem), #201 (start at 0.01), #214 (`BactrianSubtreeScale`), #218 (gain exponent), #203 / beast2#1223 (`SampleOffValues`).
- Versions: BEAST 2.7.9 Prerelease; BEAST 3 v2.8.0-rc2 (`aa8512e`), #214 at `1d7e680`, #218 at `c071a1e`; macOS, JDK 26.